

Markdown Viewer CLI – Program Specification

Author: Jan Jenkins_

Version: 3.0.0 (source of truth: ``mdutil/version.py``)

Last-Updated: 2026-07-27

License: MIT

Repository: <<https://github.com/jhenkins/mdutil>>

1. Introduction

``mdutil`` is a **cross-platform, terminal-based Markdown viewer** that renders Markdown files with syntax-highlighted code blocks and basic styling (headings, lists, blockquotes, etc.). It also supports exporting documents to PDF or HTML.

The focus is on a clean, fast, and fully-featured viewer with a few light editing functions. The program is written in Python.

Key design goals

- 100 % terminal-only (no GUI).
 - Minimal dependencies – a single static binary.
 - Cross-platform: runs on **Linux, macOS, Windows** (native).
 - Responsive UI: scrolling, line-number toggling, and theming.
 - **Extensible**: future editing features can plug into the same pipeline.
 - **Export**: convert Markdown to PDF (``--export pdf``) or HTML (``--export html``) without external tools.
-

2. Scope & Use-Cases

#	Use-Case	Description
1	View	Open any Markdown file, scroll, and read for
2	Syntax Highlight	Code blocks are highlighted using the spec
3	Theming	User can choose from built-in themes or pr
4	Line Numbers	Optional display of line numbers for code b
5	Edit	In-place Markdown editing with explicit sav

6	**Export**	Render Markdown to PDF (<code>--export pdf`</code>) or
---	-------------------	--

> ****Out-of-Scope**** (for v1.0.0)

> - Rich text editing, live preview, or file synchronization.

> - Interac

3. Functional Requirements

Feature	Description	CLI Option(s)
Load Markdown	Accept a file path or read from stdin	<code>-d <file.md>`</code> or <code><code>cat file.md &</code>
Basic Rendering	Render headings, lists, blockqu	-
Code Syntax Highlighting	Detect language via fenced code block info strings such as <code><code>``` pyth</code>	
Theming	Choose a theme.	<code>--theme <theme-name>`</code>
Custom Theme	Load a custom theme file (JSON or TOML) file <code><path>`</code>	<code>-t <THEME>file <path>`</code>
Configuration File	Load user defaults from an edit	<code>--config <path>`</code> and <code>--generate-config`</code>
Line Numbers	Toggle line numbers for code blocks	<code>--line-numbers`</code>
Scroll	Arrow keys or <code>`j/k`</code> to scroll up/down	-
Editing	Vim-like normal commands with	<code>--prompt-toolkit insert editing. Normal/view</code>
Search	Search from normal mode with	- and <code>`N`</code> , see
Safe File Writes	File-backed interactive sessions	<code>--write-through atomic same-directo</code>
Status Bar	Distinct normal/edit/dirty/error	<code>`status_bar_normal`, `status_bar_ime</code>
Help	Show command-line usage.	<code>--help`</code>
Version	Print version.	<code>--version`</code>
Export PDF	Export rendered Markdown to a PDF file using <code>fpdf2</code> (headings, paragraphs	<code>--export pdf`</code>
Export HTML	Export rendered Markdown to a	<code>--export html`</code> (same toke
Output Path	Write exported content to a specific file <code><path>`</code> or <code>-o <path>`</code>	<code>-o <path>`</code>
Output Directory	Write exported files into a direc	<code>--output-dir <dir>`</code> filename.
Multi-Format	Export to both PDF and HTML in one pass	<code>--export pdf,html`</code>
Custom CSS	Embed a user-supplied CSS file	<code>--custom-css <path>`</code> ded after b
Configuration	Export defaults are read from the <code>[export]</code> section in <code>~/.mdutilc</code> (pap	<code>[export]</code> section in <code>~/.mdutilc</code> (pap

> ****User-Interface Constraints****

> - Must be usable on terminals up to 256 color support.

> - Works i

> ****Interactive UI Library Decision****

> - v1 uses ``prompt-toolkit`` for the interactive viewer because it provides cross-platform keyboard handling, terminal resizing, fullscreen rendering, and future editing primitives while

remaining lighter than a full widget framework.

> - `textua
richer TUI v

> **Configuration File**

> - `mdutil` supports a plain-text, standard-editor-editable user configuration file for runtime defaults.

> - On Linu

4. Non-Functional Requirements

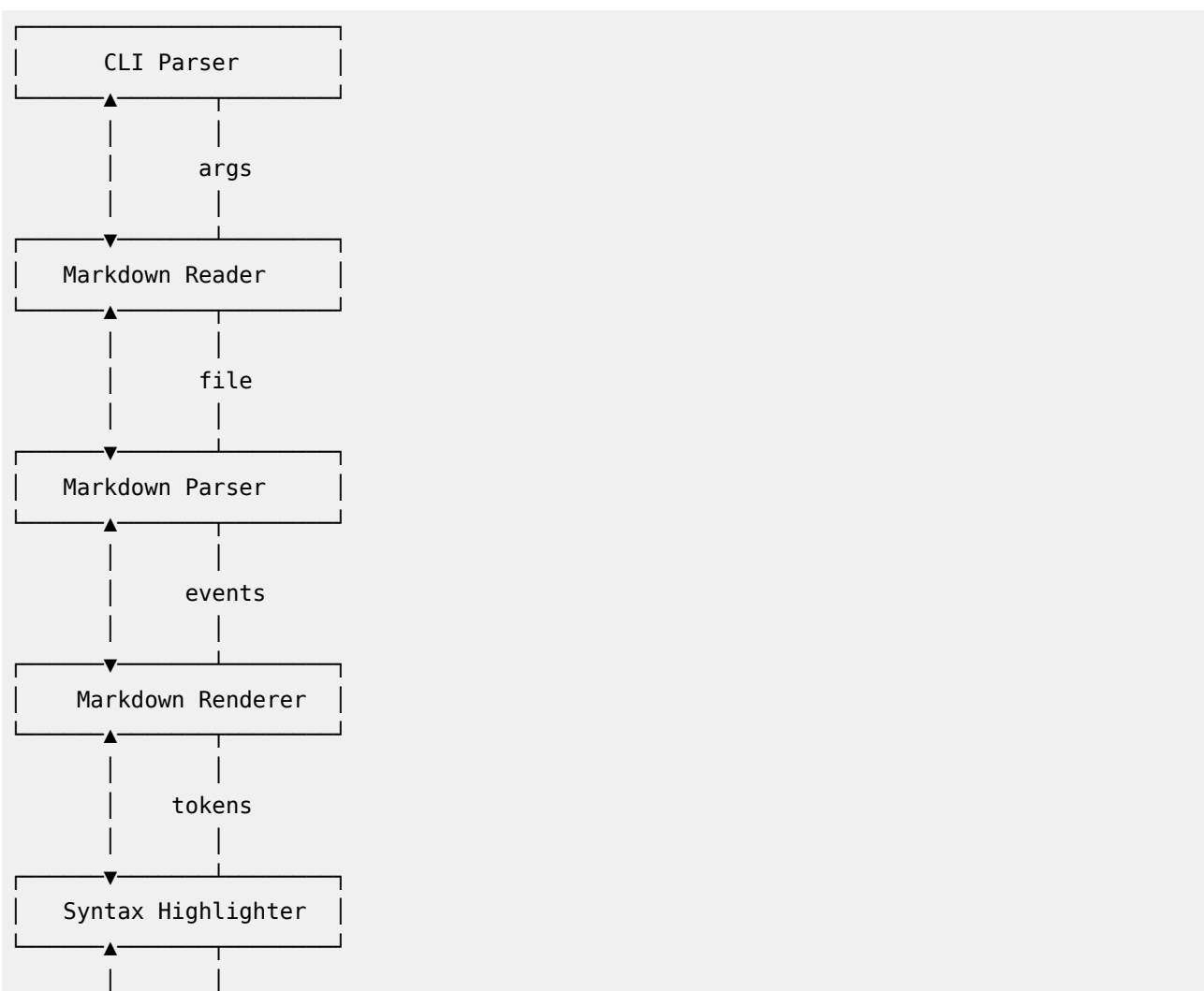
Category	Requirement	Rationale
Performance	Render < 1 ms per page load (~Fast lines)	Fast scrolling on large docs.
Memory	≤ 10 MB RAM usage on 10 k line	Lightweight on low-resource machines.
Portability	Build once, run on Linux, macOS, Windows (64-bit)	Simplified distribution.
Accessibility	High-contrast themes, `--theme`	Meets accessibility standards.
Extensibility	Modular architecture: Parser → Renderer → Plugin/Widget/Display	Resilient to plugin/widget changes.
Security	No external network access; all	Reduces attack surface.

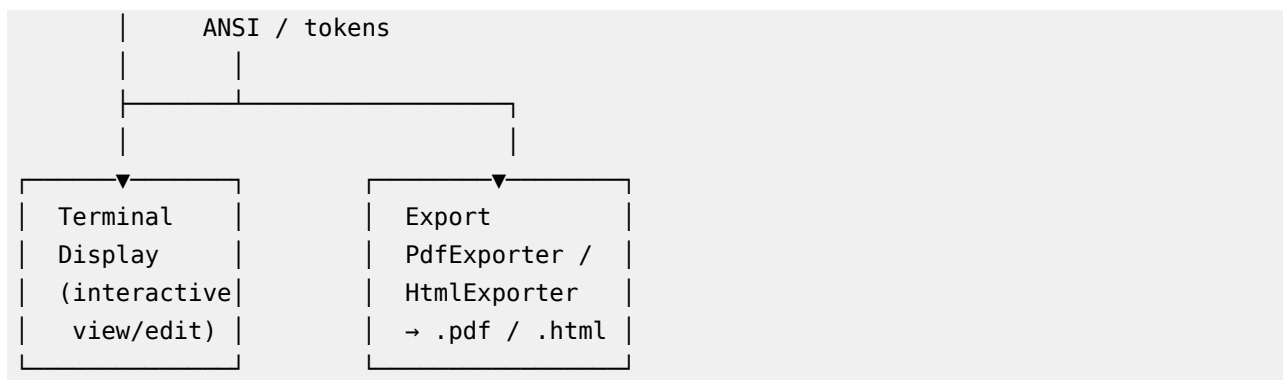
5. Architecture Overview

The CLI parser loads built-in defaults, merges any user configuration file, then applies explicit command-line options before passing resolved runtime settings through the rest of the pipeline.

Interactive terminal output has two coordinated modes. Normal/viewer mode renders

the current Markdown buffer through the parser, renderer, syntax highlighter, and terminal display pipeline. Insert/editing mode exposes the raw Markdown buffer for text changes. The command model is an explicit hybrid: Vim-like normal commands for navigation and structural editing, plus prompt-toolkit text editing while in insert mode. Normal-mode copy/paste commands are ``yc`` for the character under the cursor, ``yw`` for the word under the cursor, ``yy`` for the current line, and `Ctrl-V` to paste clipboard text at the cursor. File-backed sessions save explicitly with `Ctrl-S`, track dirty state, block accidental dirty quits, and write atomically via same-directory temporary files before replacing the target path. The bottom status bar identifies normal/insert state, save/dirty state, and the relevant safe action; its normal and insert colors can be supplied by the active theme or overridden from the configuration file. Search keys are mode-aware and appear in the bottom status bar: normal mode advertises ``/`` plus ``n/N``, while insert/editing mode advertises `Ctrl-/` so ``/`` remains normal text input.





6. Testing Strategy

Test Type	Description
Unit Tests	Each component (<code>Renderer</code> , <code>Highlighter</code>) gets pure-function tests.
Integration Tests	Run <code>mdutil</code> against a set of sample Markdown files (coverage goal).
Configuration Tests	Verify default config paths, config generation, comments/docs.
Interactive Editor Tests	Verify normal/insert mode transitions, <code>i</code> , Escape, <code>dd</code> , <code>c</code> .
Export Tests	Verify PdfExporter (bytes, paper sizes, margins, headers/footers).
Cross-Platform CI	GitHub Actions matrix: ubuntu, macos, windows.
Performance Benchmarks	Measure rendering time on large docs (10k lines).

> **Coverage Goal**: ≥ 90 % code coverage.

7. Maintenance & Contribution

- **Issue Tracker**: Use GitHub issues.
- **Pull Requests**: Follow the template, run CI, ensure tests pass.
- **Release Process**: Semantic versioning; tag releases; publish binaries on GitHub Releases and, optionally, as a Homebrew/Cargo package.

8. Future Enhancements (Roadmap)

The project roadmap is maintained as a single source of truth in `todo.md` under the "Immediate priorities" section. This spec intentionally does not duplicate the roadmap table; consult `todo.md` for current priorities, backlog items, and the housekeeping/merge state of the branch.